

04 API - Gebruik van Generics & UnitOfWork

📅 09-09-2026 ⌚ Ongeveer 21 minuten 🗨️ Ongeveer 3078 woorden

Generics

Binnen de taal C# heeft generics de betekenis "Niet voor één specifiek datatype". Dit betekent dat je code-componenten kan maken die door verschillende datatypen gebruikt kan worden. Deze kunnen dan later gebruikt worden doorheen je programma-code waarbij het datatype flexibeler in gebruik is.

Voordelen van generics

- **Herbruikbaarheid van code-componenten:** Met generics kun je dezelfde code gebruiken voor verschillende datatypen zonder duplicatie.
- **Type-safe:** Foute datatypen worden al bij het compileren gedetecteerd, waardoor runtime-fouten verminderen.
- **Performatievoordelen:** Generics vermijden boxing en unboxing, wat geheugen en verwerkingstijd bespaart.

Info

Boxing is het proces waarbij een waarde-type (zoals int) wordt omgezet naar een object, en **unboxing** is het omgekeerde: het object weer terugzetten naar het waarde-type. Dit gebeurt bijvoorbeeld als je een int opslaat in een ArrayList (die objecten verwacht). Bij elke boxing/unboxing wordt extra geheugen gebruikt en moet de runtime extra werk doen om de waarde te kopiëren en terug te converteren.

Huidige aanpak

Wanneer je tot heden CRUD-operaties van Entity Framework uitvoerde op de database, deed je dit telkens rechtstreeks in je action-methods binnen de controllers.

Voordelen van deze aanpak

- **Specifieke en eenvoudige code:** Door middel van specifieke transacties uit te voeren om gegevens op te halen of te beheren, is de code relatief eenvoudig om te schrijven. Ook draagt het bij tot meer controle over de transactie van gegevens.

Nadelen van deze aanpak

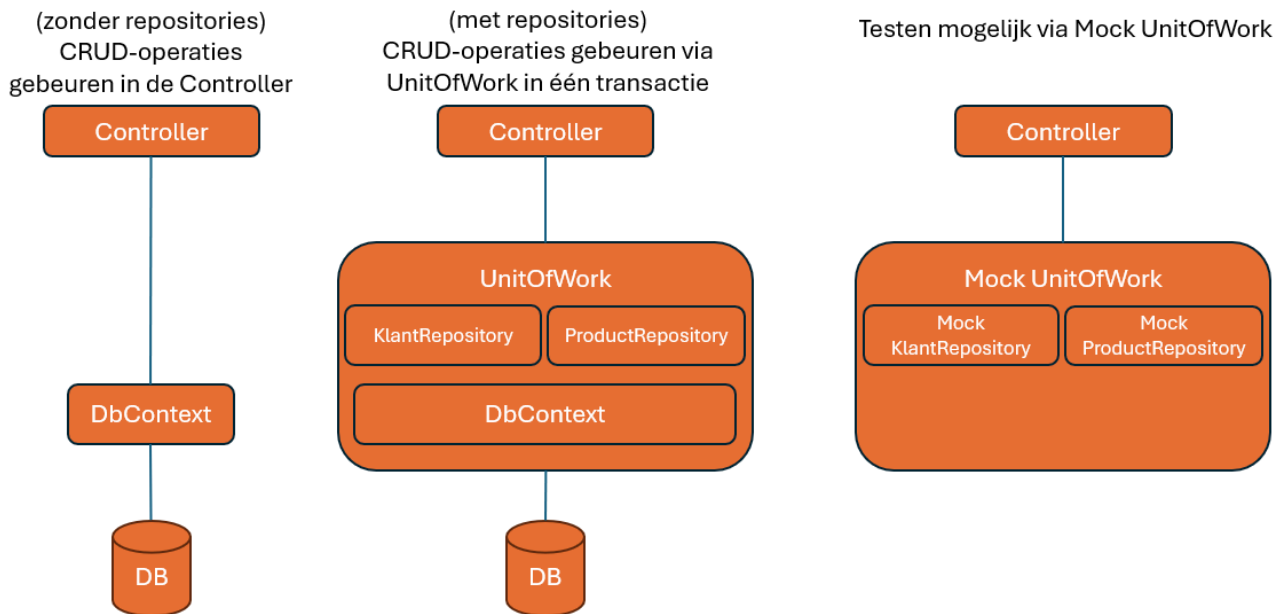
- **Mogelijks dubbele code:** Wanneer je bij elke action-method binnen je controllers de specifieke transacties van gegevens uitvoert, zal er zich veel gelijkaardige code bevinden over deze controllers heen.
- **Wijzigingen uitvoeren in elke controller:** Wanneer je dan vervolgens wijzigingen moet uitvoeren binnen deze logica, zal je dit ook op al deze plaatsen moeten gaan aanpassen. Stel dat je beslist om in de toekomst met een ander framework te werken zoals Dapper, moet je al de controllers aanpassen.
- **Geen scheiding van verantwoordelijkheid:** Door de specifieke transacties van het ophalen en beheren van gegevens binnen de action-methodes te gebruiken, is er geen scheiding tussen de businesslogica en het ophalen van data. Ook is het vervangen van het Entity Framework bij een toenemende code steeds moeilijker door de grote hoeveelheid van aanpassingen doorheen de hele codebase. De focus van Controllers moet liggen op modelstate validations en correcte http codes returnen.

Repositories

Om de problemen die voorgaande aanpak met zich meebracht te omzeilen, kan je best gebruik maken van een repository. Repositories fungeren als een scheidingslaag tussen de business logica in de controllers en de onderliggende gegevens van de database. In essentie verplaats je de logica rondom de transacties (die gegevens uit de database opvragen of beheren) van de controller naar de repository. De controllers bevatten zo enkel nog de businesslogica en de mogelijkheid om via deze repository data aan te spreken.

Deze opsplitsing van je code zorgt niet enkel voor een scheiding van verantwoordelijkheden maar brengt ook enkele andere voordelen met zich mee. Het laat je bijvoorbeeld toe om zonder grote aanpassingen alternatieve frameworks te gebruiken in plaats van Entity Framework. Verder zorgt het ook voor meer controle over de transacties die je kan uitvoe-

ren in onze controllers. Ten slotte is de code ook beter testbaar. Je kan aan de hand van repositories mocking data voorzien om zo functionaliteit binnen controllers uitbundig te testen met vooropgestelde data.



Binnen repositories kan je generieke methodes (meer info over generics op het einde van dit hoofdstuk) aanmaken binnen onze repository om zo met minder code gelijkaardige logica te voorzien. Je maakt dus een generieke repository waarin je de CRUD-operaties voorziet. Daarna kun je voor elke entity dezelfde repository gebruiken vanuit de logica binnen de controllers.

Disclaimer: Het voorbeeld binnen deze cursustekst is enkel toegespitst op logica rondom het ophalen en beheren van producten. In het demo-project van dit hoofdstuk vind je ook uitgewerkte code voor alle logica rondom de andere modellen. **UnitOfWork** is immers ook een proces uit meerdere stappen. Kort samengevat moet je volgende stappen uitvoeren:

1. Aanmaken Generieke Repository en Interface --> Generieke methoden aanmaken
2. Aanmaken Specifieke Repositories en Interfaces --> Voor methoden die niet generiek kunnen gemaakt worden zoals het opzoeken van een product op ProductNaam.
Specifieke Repositories erven over van de **IGenericRepository**
3. Aanmaken **UnitOfWork** en Interface --> Maak hier de repositories aan.
4. Maak een service aan voor **UnitOfWork** in **Program.cs**.
5. Injecteer en gebruik de **UnitOfWork** in je Controllers.

Aanmaken van Repositories

Voor het aanmaken van je repository, maak je eerst een interface aan. Deze zal als "contract" dienen en de definities van de benodigde methodes declareren. Later kan je de implementatie van deze methodes voorzien binnen de effectieve klasse. De generieke methoden worden gedeclareerd in een interface `IGenericRepository`.

```
1 public interface IGenericRepository <TEntity>
2 where TEntity : class
3 {
4     Task<IEnumerable<TEntity>> GetAllAsync();
5     Task<TEntity?> GetByIdAsync(int id);
6     Task AddAsync(TEntity entity);
7     void Update(TEntity entity);
8     void Delete(TEntity entity);
9 }
```

Daarna werk je de `GenericRepository` klasse uit. Deze erft over van de interface en werkt een implementatie uit voor de eerder gedeclareerde methodes. Om onderstaande CRUD-operaties uit te voeren op je database heb je een databasecontext nodig. Die moet je toevoegen aan de hand van dependency injection via de constructor.

Verwachte methodes

- **GetAllAsync:** Je zal een lijst van gegevens ophalen.
- **GetByIdAsync:** Je zal één record ophalen aan de hand van een Id.
- **AddAsync:** Hierbinnen zal je één record kunnen toevoegen.
- **Update:** Je zal één record kunnen wijzigen aan de hand van een meegegeven Id.
- **Delete:** Je zal één record kunnen verwijderen aan de hand van een meegegeven Id.

```
1 public class GenericRepository <TEntity> : IGenericRepository<TEntity>
2 where TEntity : class
3 {
4     //Dependency Injection context
5     private readonly WebAPIDemoContext _context;
6
7     public GenericRepository(WebAPIDemoContext context)
8     {
```

```
9         _context = context;
10     }
11
12     // implementaties
13     public async Task<IEnumerable<TEntity>> GetAllAsync()
14     {
15         return await _context.Set<TEntity>().ToListAsync();
16     }
17
18     public async Task<TEntity?> GetByIdAsync(int id)
19     {
20         return await _context.Set<TEntity>().FindAsync(id);
21     }
22
23     public async Task AddAsync(TEntity entity)
24     {
25         await _context.Set<TEntity>().AddAsync(entity);
26     }
27
28     public void Update(TEntity entity)
29     {
30         _context.Set<TEntity>().Update(entity);
31     }
32
33     public void Delete(TEntity entity)
34     {
35         _context.Set<TEntity>().Remove(entity);
36     }
37
38 }
```

Je kunt voor de `IGenericRepository` een service in `Program.cs` aanmaken en deze service gebruiken in je Controllers. Ook voor specifieke Repositories die overerven van de Generieke repositories kun je hetzelfde doen. Je gaat dan uiteindelijk veel services aanmaken en je moet die telkens één voor één gaan injecteren in je controllers waardoor het moeilijk wordt om de code te onderhouden.

UnitOfWork

Om deze nieuwe problemen te voorkomen, kan je best gebruik maken van UnitOfWork. UnitOfWork fungeert namelijk als een extra component dat het overzicht biedt over al de repositories, die je doorheen de applicatie zal gebruiken. In essentie ontvang je een gemeenschappelijke context en beheer je een collectie van repositories. Daardoor kan je een reeks van commando's uitvoeren die je vervolgens kan afsluiten door alle wijzigingen op te slaan.

Deze opsplitsing van je code zorgt niet enkel voor een gemeenschappelijke context, maar brengt ook enkele andere voordelen met zich mee. Het laat je ook toe om enkel nog UnitOfWork te registreren voor Dependency Injection binnen je `Program.cs`. Daardoor vervallen alle registraties van andere repositories dus. Verder kan je ook `IUnitOfWork` gebruiken om een afzonderlijke testversie van UnitOfWork te maken die repositories beheert. Deze testversie dient uitsluitend om testdata te voorzien en zo de testbaarheid van de applicatie te verhogen.

Zoals je merkt, biedt UnitOfWork heel wat voordelen. Helaas maakt het de code-base een stuk complexer en heeft de data-laag zo ook een hogere leercurve. Daarom is het belangrijk om af te wegen of UnitOfWork een meerwaarde biedt voor de applicatie. Kleine applicaties zijn vaak sneller gemaakt zonder een UnitOfWork of repositories te voorzien.

Aanmaken van UnitOfWork

Voor het aanmaken van UnitOfWork, maak je eerst een interface aan. Deze zal als "contract" dienen en de definities van de benodigde properties en methodes declareren. Later kan je de implementatie van deze properties en methodes voorzien binnen de UnitOfWork-klasse.

```
1 public interface IUnitOfWork csharp
2 {
3     IGenericRepository<Product> ProductRepository { get; }
4
5     public void SaveChanges();
6 }
```

Vervolgens werk je de UnitOfWork klasse verder uit. Deze zal je interface gebruiken en voor de eerder gedeclareerde properties en methodes een implementatie verwachten. Doorheen je UnitOfWork maak je gebruik van de eerder gemaakte repositories die je gegevens kunnen opvragen of beheren. Hiervoor heb je een context nodig, die je zal toevoegen aan de hand van dependency injection via de constructor. Deze context wordt vervolgens voor elke repository van dezelfde context te voorzien.

Info

De kern van het Unit of Work-patroon is dat alle wijzigingen aan verschillende entiteiten in één transactie gebeuren. Je geeft de databasecontext (WebAPIDemoContext) mee aan je UnitOfWork zodat alles wat je binnen één request doet, met dezelfde verbinding en hetzelfde geheugenbeeld van de database werkt. Stel je voor dat meerdere repositories (zoals Producten en Bestellingen) tegelijk iets aanpassen: als die elk hun eigen context zouden gebruiken, zou je wijzigingen niet netjes in één transactie kunnen opslaan en kan je data inconsistent worden.

```
1 public class UnitOfWork : IUnitOfWork
2 {
3     private readonly WebAPIDemoContext _context;
4
5     private IGenericRepository<Product> productRepository;
6
7     public UnitOfWork(WebAPIDemoContext context)
8     {
9         _context = context;
10    }
11
12    public IGenericRepository<Product> ProductRepository
13    {
14        get
15        {
16            if (this.productRepository == null)
17                this.productRepository
18                    = new GenericRepository<Product>(_context);
19            return productRepository;
20        }
21    }
22    // Verkorte schrijfwijze met de null-coalescing assignment operator
23    // (?=). Als productRepository null is, ken er dan de nieuwe waarde aan
24    // toe.
25    //public IGenericRepository<Product> ProductRepository =>
```

```
25     productRepository ??= new GenericRepository<Product>(_context);
26
27
28     //Onderstaande methode kan ook met async/await en SaveChangesAsync
29     public void SaveChanges()
30     {
31         _context.SaveChanges();
32     }
33 }
```

Om UnitOfWork te kunnen gebruiken in je controllers, moet je deze eerst aan de services toevoegen in Program.cs . Deze zorgt ervoor dat de dependency injection van UnitOfWork vlot verloopt wanneer deze aangeroepen wordt.

Aangezien UnitOfWork de onderliggende repositories beheert en voorziet van één en dezelfde context, is het enkel noodzakelijk om UnitOfWork toe te voegen aan de services. Alle andere repositories kunnen dan verwijderd worden uit de services.

1

```
builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();
```

csharp

Gebruik van UnitOfWork

Momenteel werkt je code-base nog zoals voorheen, je maakt immers nog geen gebruik van de UnitOfWork-klasse die je aangemaakt hebt. Om gebruik te kunnen maken hiervan, haal je eerst de repository binnen via dependency injection via de constructor van de desbetreffende controller.

```
1
2
3
4
5
6
7
8
9
```

```
public class ProductController : ControllerBase
{
    private readonly IUnitOfWork _unitOfWork;

    public ProductController(IUnitOfWork unitOfWork)
    {
        _unitOfWork = unitOfWork;
    }
}
```

csharp

Vervolgens kan je in elke action-method van de controller de code om de repository aan te roepen, vervangen door de benodigde code van de UnitOfWork-klasse. Hieronder vind je enkele voorbeelden van action-methods waarbinnen UnitOfWork gebruikt wordt.

```
1 public class ProductController : ControllerBase
2 {
3     //Dependency Injection UnitOfWork
4
5     [HttpGet("GetAll")]
6     public async Task<ActionResult<List<Product>>> Get()
7     {
8         var producten = await
9         _unitOfWork.ProductRepository.GetAllAsync();
10         if (producten.ToList().Count > 0) {
11             return Ok(producten);
12         }
13         else
14         {
15             return NotFound("Er zijn geen resultaten gevonden");
16         }
17     }
18 }
```

csharp

```
1 public class ProductController : ControllerBase
2 {
3     //Dependency Injection UnitOfWork & voorgaande methode
4
5     [HttpGet("{id}")]
6     public async Task<ActionResult<Product>> GetProduct(int id)
7     {
8         Product? product = await
9         _unitOfWork.ProductRepository.GetByIdAsync(id);
10         if (product == null)
11         {
12             return NotFound(
13                 $"Product {id} kan niet worden gevonden in de database");
14         }
15         return Ok(product);
16     }
17 }
```

csharp

```
1 public class ProductController : ControllerBase
2 {
3     //Dependency Injection UnitOfWork & voorgaande methodes
4
5     [HttpPost]
```

csharp

```
6         public async Task<ActionResult<Product>> ProductToevoegen(Product
7         product)
8         {
9             var producten = await
10         _unitOfWork.ProductRepository.GetAllAsync();
11
12             if (producten == null)
13                 return NotFound("De tabel Producten bestaat niet in de
14         database.");
15             if (!ModelState.IsValid)
16             {
17                 return BadRequest(ModelState);
18             }
19
20             await _unitOfWork.ProductRepository.AddAsync(product);
21             try
22             {
23                 _unitOfWork.SaveChanges();
24             }
25             catch (DbUpdateException dbError)
26             {
27                 return BadRequest(dbError);
28             }
29             return CreatedAtAction("GetProduct", new {id = product.Id},
30         product);
31         }
32     }
```

Conclusie

UnitOfWork heeft verschillende voordelen: je hoeft slechts één context te voorzien aan de UnitOfWork, die deze verder zal gebruiken in de onderliggende repositories. Zo kan je "batches" aan bewerkingen uitvoeren over verscheidene entities heen vooraleer je alles in één keer opslaat. Ook hoeft je maar éénmaal te registreren als een service voor je dependency injection toepast.

Toch zijn er nog zaken die beter kunnen. Wanneer je bijvoorbeeld een methode wil uitvoeren die afwijkt van de 5 methodes die in de generieke repository voorzien zijn, zal je afwijkende code moeten voorzien. Een voorbeeld hiervan is onderstaande code.

```
1 public class ProductController : ControllerBase
2 {
3     //Dependency Injection UnitOfWork & voorgaande methodes
4
5     [HttpGet("Search")]
6     public async Task<ActionResult<List<Product>>> Search(string
7 zoekwaarde)
8     {
9         var producten = await
10         _unitOfWork.ProductRepository.GetAllAsync();
11         var product =
12
13         producten.Where(x=>x.Naam.Contains(zoekwaarde)).OrderBy(x=>x.Naam);
14         if (product == null)
15         {
16             return NotFound("Er zijn geen producten in de database" +
17                 $"waar {zoekwaarde} voorkomt in de naam");
18         }
19         return Ok(product);
20     }
21 }
```

csharp

Merk op dat je eerst de volledige lijst van alle producten binnenhaalt. Vervolgens ga je deze lijst dan filteren op producten met een naam die de zoekwaarde bevat. Wanneer je met grote hoeveelheden aan gegevens werkt, is dit niet meer rendabel. Je wilt daarom enkel de gegevens ophalen die voor ons relevant zijn.

Specifieke repositories

Om specifieke en relevante gegevens op te halen, kan je best gebruik maken van een specifieke repository. Deze kan met parameters werken die afwijken van de generieke repository die je eerder maakte. Specifieke repositories erven over van de generieke repository maar bieden dus bijkomende functionaliteit. Je kan de generieke repository binnen de UnitOfWork dan vervangen door deze specifieke repository.

Aanmaken van specifieke repositories

Om te kunnen garanderen dat je subclasses gebruik kunnen maken van de context die je momenteel al voorziet in de generieke repository, maak je deze protected. Zo kunnen de nieuwe methodes die je maakt in de specifieke repositories ook gebruik maken van de

context in de generieke.

```

1  public class GenericRepository<TEntity> : IGenericRepository<TEntity> csharp
2  where TEntity : class
3  {
4      protected readonly WebAPIDemoContext _context;
5  }

```

Voor het aanmaken van een specifieke repository, maak je eerst een interface aan. Deze zal als "contract" dienen en de definities van de benodigde properties en methodes declareren. Later kan je de implementatie van deze properties en methodes voorzien binnen de effectieve klasse. Bijzonder aan deze interface, ten opzichte van voorgaande interfaces die je voorzien hebt, is dat deze overerft van een andere interface. Via de generieke interface die je reeds gemaakt hebt, kan je bestaande methodes overerven en moet je deze niet meer voorzien in deze interface. Enkel uitbreidende methodes moet je nog definiëren. **Belangrijk is wel dat je aangeeft welk datatype je gebruikt binnen de generieke repository.** In dit geval werk je de ProductRepository uit, dus geef je Product als datatype mee als parameter.

```

1  public interface IProductRepository : IGenericRepository<Product> csharp
2  {
3      Task<IEnumerable<Product>> SearchAsync(string zoekwaarde);
4  }

```

Werk de repositoryklasse uit door de interface te implementeren en te erven van de generieke repository. Zo krijg je de basis-CRUD-methodes al mee. Voor de extra methodes die je eigen interface definieert, schrijf je zelf een implementatie en gebruik je daarbij de DbContext die je via de constructor hebt meegekregen.

```

1  public class ProductRepository : GenericRepository<Product>, csharp
2  IProductRepository
3  {
4      public ProductRepository(WebAPIDemoContext context) : base(context) {
5      }
6
7      public async Task<IEnumerable<Product>> SearchAsync(string
8  zoekwaarde)
9      {
10         return await _context
11             .Producten
12

```

```
13         .Where(x => x.Naam.Contains(zoekwaarde))
           .OrderBy(x => x.Naam)
           .ToListAsync();
    }
}
```

Gebruik van specifieke repositories

Door gebruik te maken van overerving, bevat de specifieke repository zowel de generieke methodes als de specifieke methodes behorende tot een bepaalde entiteit. Hierdoor kan je de generieke interface die gebruikt wordt binnen UnitOfWork ook vervangen door de specifieke repository. Je doet dit eerst in de interface van UnitOfWork om de definitie van de methode reeds in orde te maken.

```
1 public interface IUnitOfWork
2 {
3     IProductRepository ProductRepository { get; }
4 }
```

csharp

Ook binnen de klasse zelf zal je de vervanging moeten uitvoeren.

```
1 public class UnitOfWork : IUnitOfWork
2 {
3     private IProductRepository productRepository;
4
5     public IProductRepository ProductRepository
6     {
7         get
8         {
9             if (this.productRepository == null)
10                this.productRepository
11                    = new ProductRepository(_context);
12                return productRepository;
13            }
14        }
15 }
```

csharp

Op zich zal dat in je Controller niet veel veranderen, behalve dat de nieuwe methoden nu beschikbaar zijn in je UnitOfWork.

```
1 [HttpGet("Search")]
2 public async Task<ActionResult<List<Product>>> Search(string zoekwaarde)
3 {
4     // var producten = await _unitOfWork.ProductRepository.GetAllAsync();
5     // var product =
6     producten.Where(x=>x.Naam.Contains(zoekwaarde)).OrderBy(x=>x.Naam);
7     var producten = await
8     _unitOfWork.ProductRepository.SearchAsync(zoekwaarde);
9
10    if (producten == null)
11    {
12        return NotFound($"Er zijn geen producten in de database waar
13        {zoekwaarde} voorkomt in de naam");
14    }
15    return Ok(producten);
16 }
```

Extra voorbeelden van Generics

Hieronder zijn nog enkele voorbeelden die illustreren hoe generieke variabelen en methodes gebruikt kunnen worden. Deze zijn afzonderlijk van de voorbeeldoefening en dienen enkel ter illustratie.

Voorbeeld 1: Methodes met generieke variabelen

Dit voorbeeld zal binnen een generieke methode ook gebruik maken van generieke variabelen. Dit betekent dat wanneer je een type meegeeft als parameter bij de methode, de variabele gaan fungeren als dat specifieke type dat meegegeven werd.

```
1 private static void Voorbeeld1() {
2     int getal1 = 1;
3     int getal2 = 2;
4
5     Afdrukken("Voor wissel", getal1, getal2);
6     Omwisselen(ref getal1, ref getal2);
7     Afdrukken("Na wissel", getal1, getal2);
8
9     string waarde1 = "a";
10    string waarde2 = "b";
```

```
11
12     Afdrukken("Voor wissel", waarde1, waarde2);
13     Omwisselen(ref waarde1, ref waarde2);
14     Afdrukken("Na wissel", waarde1, waarde2);
15 }
```

```
1     private static void Afdrukken(string titel, int x, int y) {           csharp
2         Console.WriteLine(titel);
3         Console.WriteLine(x);
4         Console.WriteLine(y);
5     }
6
7     private static void Afdrukken(string titel, string x, string y) {
8         Console.WriteLine(titel);
9         Console.WriteLine(x);
10        Console.WriteLine(y);
11    }
```

```
1     private static void Omwisselen(ref int x, ref int y)                csharp
2     {
3         int temp = x;
4         x = y;
5         y = temp;
6     }
7
8     private static void Omwisselen(ref string x, ref string y)
9     {
10        string temp = x;
11        x = y;
12        y = temp;
13    }
```

```
1     private static void Afdrukken<T>(string titel, T x, T y)           csharp
2     {
3         Console.WriteLine(titel);
4         Console.WriteLine(x);
5         Console.WriteLine(y);
6     }
```

```
1 private static void Omwisselen<T>(ref T x, ref T y) csharp
2 {
3     T temp = x;
4     x = y;
5     y = temp;
6 }
```

```
1 private static void Voorbeeld1() { csharp
2     int getal1 = 1;
3     int getal2 = 2;
4
5     Afdrukken("Voor wissel", getal1, getal2);
6     Omwisselen(ref getal1, ref getal2);
7     Afdrukken("Na wissel", getal1, getal2);
8
9     string waarde1 = "a";
10    string waarde2 = "b";
11
12    Afdrukken("Voor wissel", waarde1, waarde2);
13    Omwisselen(ref waarde1, ref waarde2);
14    Afdrukken("Na wissel", waarde1, waarde2);
15 }
```

Voorbeeld 2: Methode met generieke lijst

Dit voorbeeld zal binnen een generieke methode ook gebruik maken van generieke lijsten. Dit betekent dat wanneer je een lijst meegeeft als parameter bij de methode, er ook een type verwacht wordt bij deze lijst. De lijst zal dan binnen deze methode fungeren alsof het van dat specifiek type is.

```
1 private static void Afdrukken() csharp
2 {
3     List<Product> producten = _context.Producten.ToList();
4     List<Klant> klanten = _context.Klanten.ToList();
5
6     AfdrukkenLijst(producten);
7     AfdrukkenLijst(klanten);
8 }
```

```
1 private static void AfdrukkenLijst(List<Product> lijst) csharp
2 {
3     foreach(var props in typeof(Product).GetProperties())
4     {
5         Console.Write(props.Name + ";");
6     }
7     Console.WriteLine();
8
9     foreach(var item in lijst)
10    {
11        foreach (var props in typeof(Product).GetProperties())
12        {
13            Console.Write(props.GetValue(item) + ";");
14        }
15        Console.WriteLine();
16    }
17 }
```

```
1 private static void AfdrukkenLijst(List<Klant> lijst) csharp
2 {
3     foreach(var props in typeof(Klant).GetProperties())
4     {
5         Console.Write(props.Name + ";");
6     }
7     Console.WriteLine();
8
9     foreach(var item in lijst)
10    {
11        foreach (var props in typeof(Klant).GetProperties())
12        {
13            Console.Write(props.GetValue(item) + ";");
14        }
15        Console.WriteLine();
16    }
17 }
```

```
1 private static void AfdrukkenLijst<T>(List<T> lijst) csharp
2 {
3     foreach(var props in typeof(T).GetProperties())
4     {
5         Console.Write(props.Name + ";");
6     }
```

```
7      Console.WriteLine();
8
9      foreach(var item in lijst)
10     {
11         foreach (var props in typeof(T).GetProperties())
12         {
13             Console.Write(props.GetValue(item) + ";");
14         }
15         Console.WriteLine();
16     }
17 }
```

```
1  private static void Afdrukken()                                csharp
2  {
3      List<Product> producten = _context.Producten.ToList();
4      List<Klant> klanten = _context.Klanten.ToList();
5
6      AfdrukkenLijst(producten);
7      AfdrukkenLijst(klanten);
8  }
```

[Hier vind je het WebApiDemo-project.](#)

Laatst bijgewerkt: 09-09-2026, 08:52

Bijdragers: JorenSynaeveTM